

Bill Bloom

(React Lifecycle, Styling, Lists & Keys, useEffect)

Hello learner!

In the previous week, you built the foundational UI components for the Bill Bloom application using React. You created reusable components, handled forms, and managed state using dummy data.

Now, it's time to enhance your application by making it more **interactive, visually appealing, and data-driven**.

Background

Your application currently has basic UI components but lacks good styling, structured data display, and lifecycle-driven behavior. The expense ledgers are placeholders, and there is no meaningful visualization of financial data.

This week, you will:

- Apply **React Bootstrap styling** to improve UI/UX
- Replace placeholders with **real data-driven components**
- Use **useEffect** to simulate data fetching
- Introduce **data visualization (graphs & charts)** from recharts for insights
- Implement **component lifecycle awareness**

Your application will now begin to feel like a **real-world financial dashboard**.

Problem Statement

Your task this week is to enhance the frontend by adding styling, lifecycle methods, data rendering, and analytics visualization.

You must:

- Style all components using React Bootstrap
- Replace placeholder sections with functional ledgers
- Implement settlements and payment flows
- Use **useEffect** for data loading and lifecycle handling
- Display analytics using charts and graphs
- Introduce navigation via a Home component

By the end of this week, your application will not only display data but also **present meaningful insights visually and interactively**.

1. Project Setup & Bootstrap Integration

Install and configure Bootstrap:

```
npm install react-bootstrap bootstrap
```

Import Bootstrap in main.jsx:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

3. Home Component & Navigation

- In the **HomePage**, include a navigation bar with options: **Home, Groups, Personal**
- On clicking:
 - **Home** → Display a simple welcome or dummy message
 - **Groups** → Display the GroupList component
 - **Personal** → Display the PersonalExpensePage component

Use **state + conditional rendering** to switch between views (no routing yet).

4. Update Styles

A. Styling Layout Components

- **Header** → Convert into a Bootstrap Navbar with branding and navigation
- **Footer** → Style using Container with centered text and subtle background

B. Styling Authentication Components

- Use Bootstrap Form components (Form, Form.Group, Form.Control)
- Apply validation styles using **isInvalid** and feedback messages
- Style buttons and spacing for better UX

C. Styling Group Components

- **GroupCard** → Use Bootstrap Card with title, content, and badges
- **GroupList** → Display cards using responsive grid layout
- **CreateGroupForm** → Styled form with checkbox inputs
- **GroupDetail** → Use Card + ListGroup for structured display

D. Expense Components Enhancement

- Style PersonalExpenseForm and GroupExpenseForm using Bootstrap
 - Use dropdowns (Form.Select) and checkbox groups (Form.Check)
 - Ensure clean spacing and layout consistency
-

5. Expense Ledgers (Replacing Placeholders)

A. Group Expense Ledger

Create a table displaying:

- Description, Amount (₹), Paid By, Category, Date, Action
- Show all the group expenses.
- Enable the Delete button on every row only for the group creator.
- Use `expense_id` as key

Expense Ledger + Add Expense									
#	Description	Amount (₹)	Date	Paid By	alice_wonder	bob_builder	carol_joy	Category	Action
1	Beach shack dinner	₹4,500	05 Mar 2024	alice_wonder	₹1,500	₹1,500	₹1,500	Food	Delete
2	Cab to Goa airport	₹6,000	06 Mar 2024	bob_builder	₹2,000	₹2,000	₹2,000	Transport	Delete
3	Water sports activity	₹3,000	07 Mar 2024	carol_joy	₹1,000	₹1,000	₹1,000	Entertainment	Delete
Total		₹13,500			₹4,500	₹4,500	₹4,500		

B. Personal Expense Ledger

- Create a similar table without "Paid By".
- Show all the personal expenses from the current login user.
- Enable the Delete button on every row.

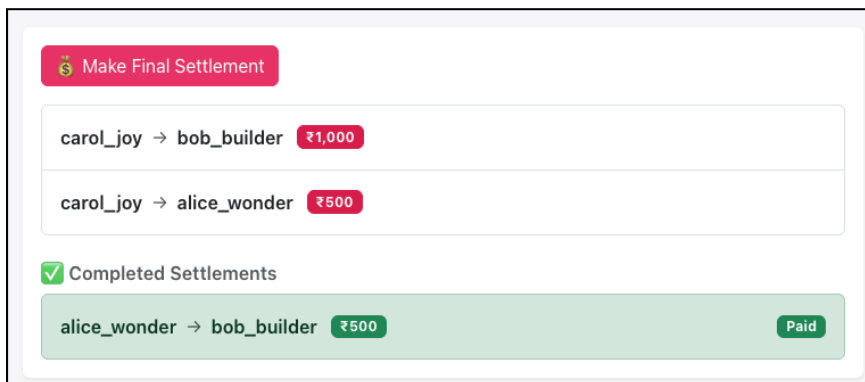
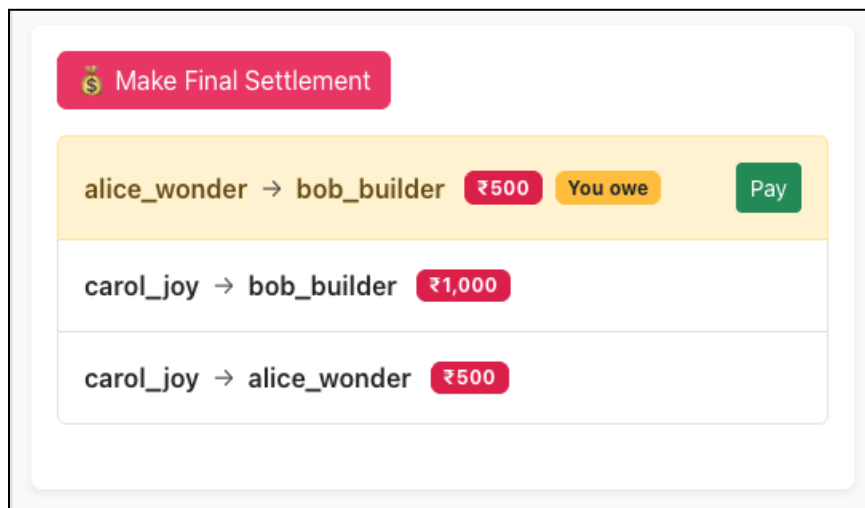
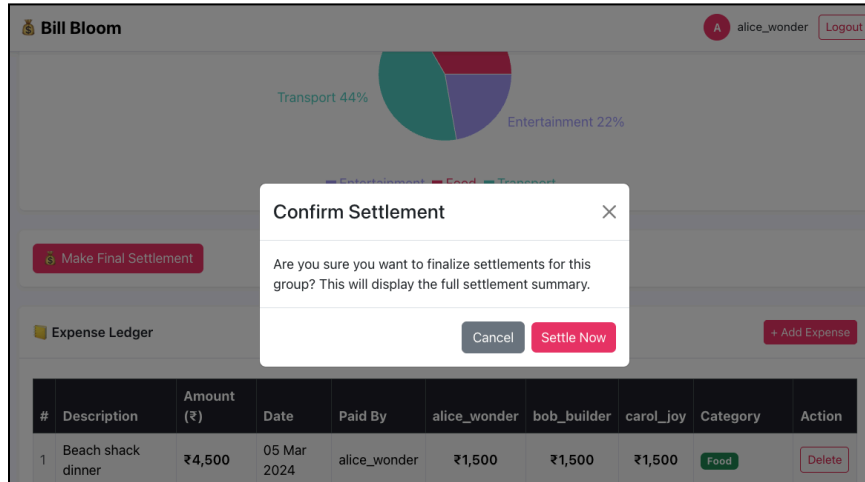
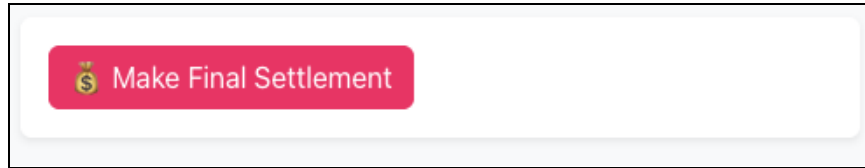
6. Settlement Components

A. Settlement Summary

Display settlements in a readable format:

User A → User B ₹XXX

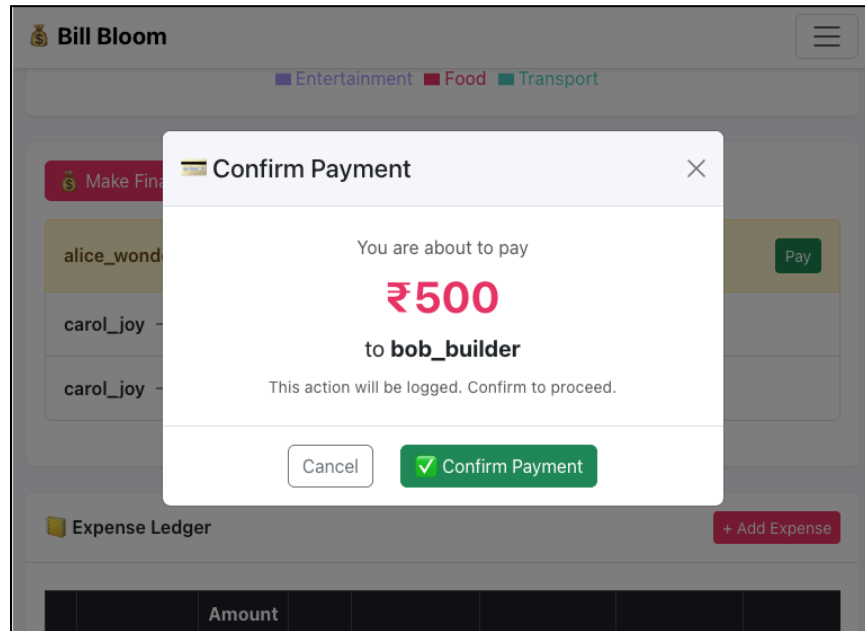
- Show "Pay" button only if logged-in user owes money
- Trigger callback on click



B. Pay Confirmation

Create a modal or overlay:

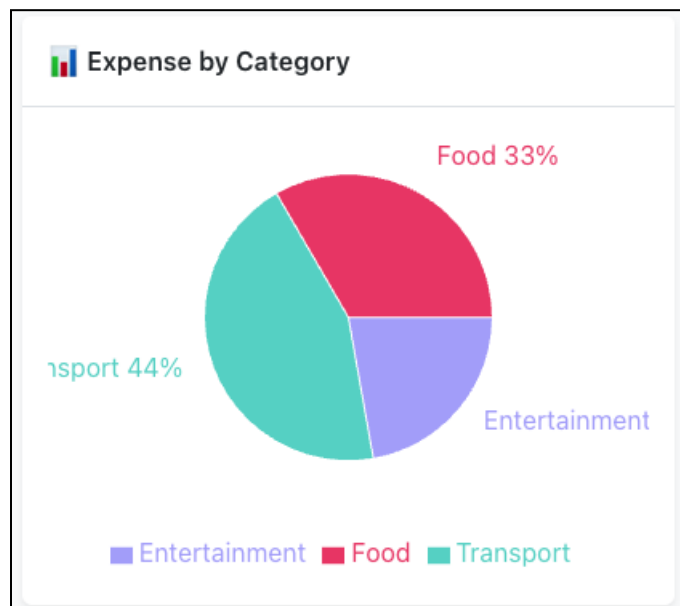
- Show payment details
- Confirm and cancel actions
- On confirm → log action to console



7. Analytics & Visualization

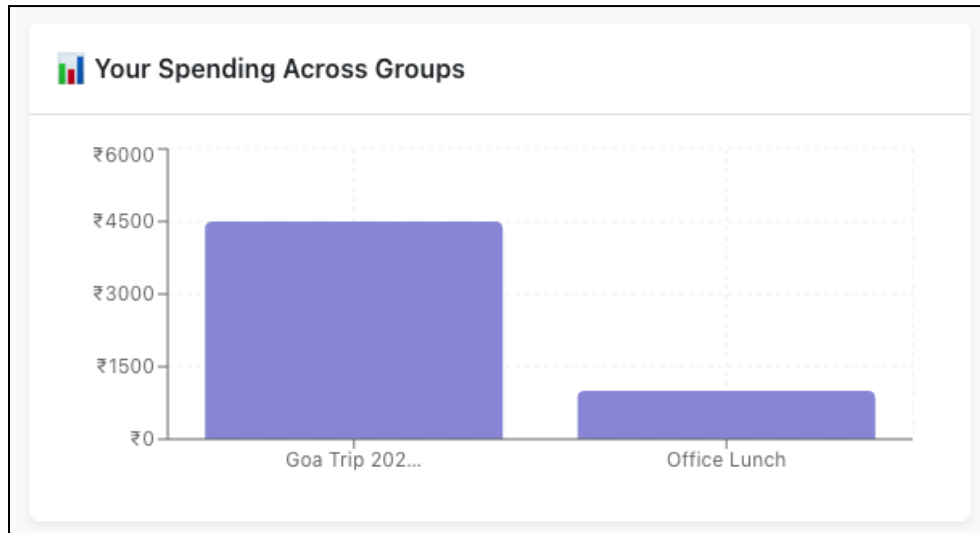
A. Group Level Analytics (GroupDetail)

- Add a **pie chart** showing expense distribution by category



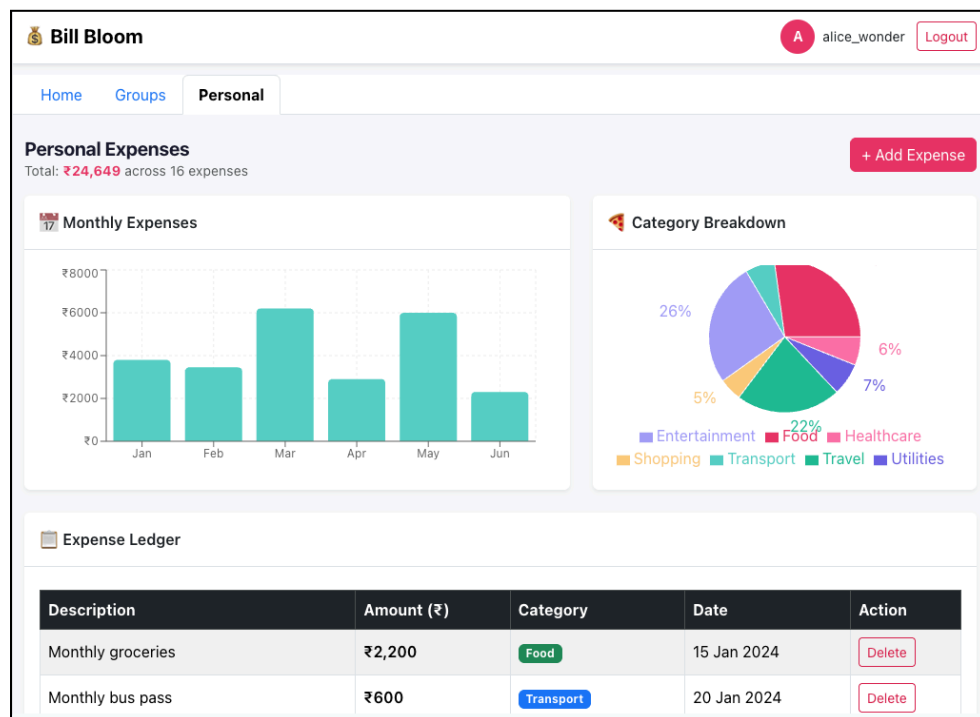
B. User Group Analytics (Groups Section)

- Add a **vertical bar graph** showing the logged-in user's spending across groups



C. Personal Analytics (Personal Expense Page)

- Add:
 - **Vertical bar graph** showing monthly expenses
 - **Pie chart** showing category-wise expense distribution



8. useEffect & Lifecycle Handling

Use useEffect to simulate API calls:

- Load data on component mount
- Use setTimeout to simulate delay
- Show loading spinner during fetch

Example:

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    setData(dummyData);  
    setLoading(false);  
  }, 800);  
  
  return () => clearTimeout(timer);  
}, []);
```

Also demonstrate dependency-based execution:

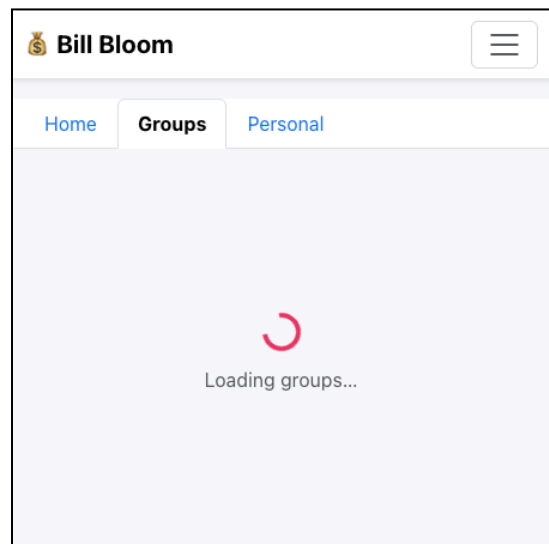
```
useEffect(() => {  
  // runs when groupId changes  
}, [groupId]);
```

9. Loading, Empty & Feedback States

- Show **Spinner** while loading
- Show **Alert messages** for empty data
- Show **confirmation dialog** before delete
- Show success/error alerts for actions

10. Integration

- Replace all ledger placeholders with actual components.
- Integrate Settlement Summary and Payment Confirmation modals into Group Detail.
- Integrate Personal Expense Ledger into Personal Expense Page.
- Ensure state updates correctly after actions.
- Check for any errors in the console and resolve them.



Happy Coding
